

# INTRODUCTION TO SCALA, SCALA REPL, AND EXPANDED FUNCTION FORMAT



by  
**Dr M Rajasekhara Babu**

School of  
Computer Science and Engineering  
<https://www.rajababu.org>

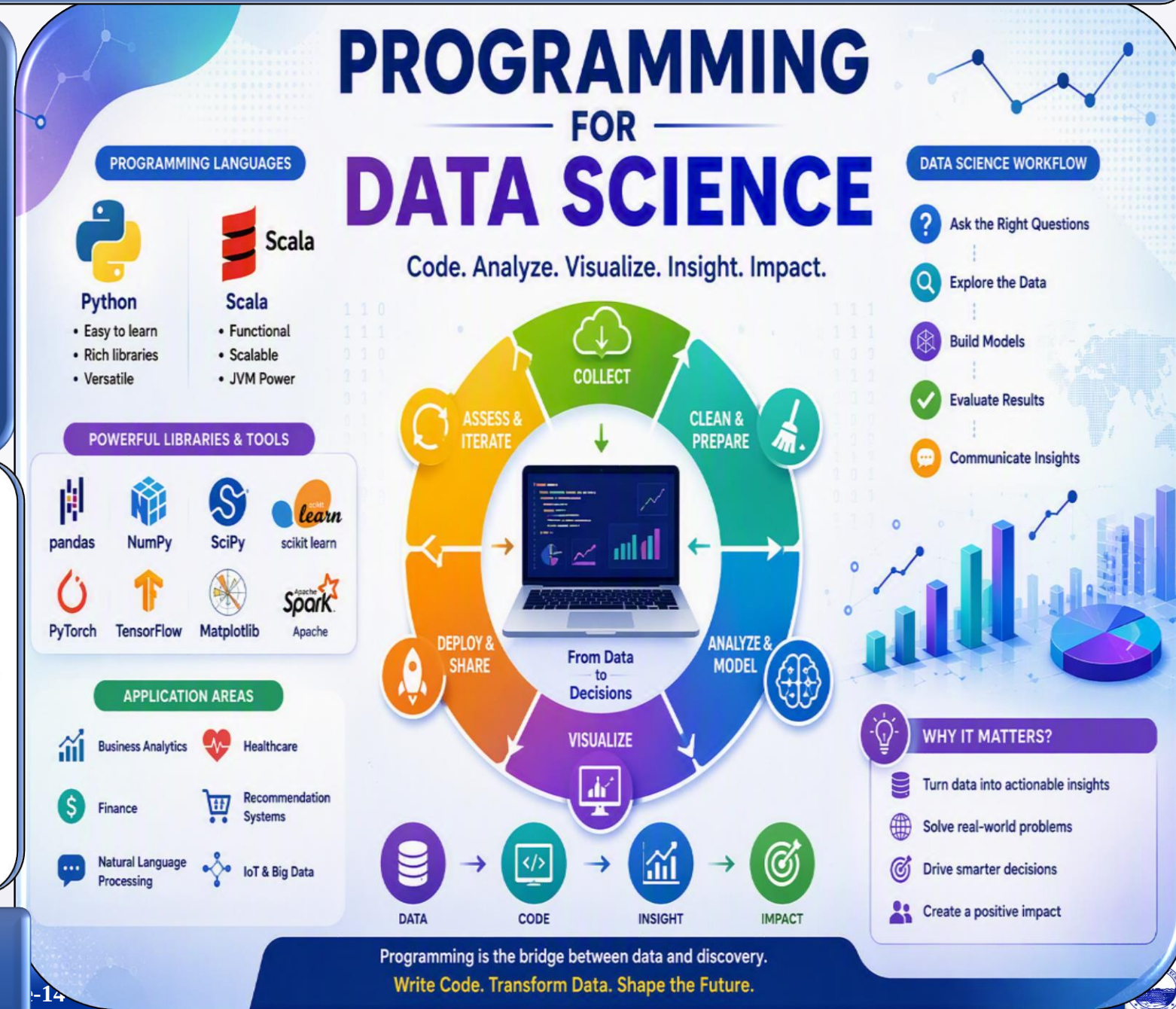


# VIT<sup>®</sup>

## Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



## Scalable Language

- Scala is designed to combine the power of
  - Object-Oriented Programming (OOP) and
  - Functional Programming (FP)in a single, unified language.
- Scala runs on the Java Virtual Machine (JVM) and can use Java libraries directly.

## Why Scala?

- Concise and expressive syntax
- Strong type system
- Supports functional programming
- Fully compatible with Java
- Widely used in Big Data (Apache Spark)

# Applications of Scala

## **Big Data Analytics**

Powering large-scale data processing pipelines.

## **Apache Spark**

The leading framework for distributed data computation.

## **Web Applications**

Building scalable, high-performance backends.

## **Machine Learning**

Constructing end-to-end ML pipelines at scale.

## Object-Oriented

- Everything in Scala is an object — even integers.
- Example: `val x = 10` treats the integer as an object.

## Functional Programming

- Functions can be stored in variables and passed as parameters.
- Example: `val square = (x:Int) => x*x`

## Type Inference

- Scala automatically identifies data types.
- `val age = 20` is equivalent to `val age:Int = 20`.

## Java Interoperability

Java libraries can be used directly inside Scala programs with no extra configuration.

Before writing Scala programs, ensure the following software is installed and verified.

| Component                  | Purpose                       | Verify Command       |
|----------------------------|-------------------------------|----------------------|
| Java Development Kit (JDK) | Required runtime for the JVM  | java -version        |
| Scala Compiler             | Compiles .scala source files  | scala -version       |
| Compile Program            | Produces bytecode from source | scalac program.scala |
| Run Program                | Executes compiled program     | scala ProgramName    |

## What is REPL?

Read → Evaluate → Print → Loop

- REPL allows programmers to execute Scala statements interactively without compiling a full program.
- Launch it by typing `scala` in the terminal.

## Advantages

- Immediate feedback on every statement
- Quick testing of code snippets
- Ideal for learning Scala
- No need to compile programs

Once launched with `scala`, you can type expressions and see results instantly:

```
scala> val a = 10  
a: Int = 10
```

```
scala> a + 20  
res0: Int = 30
```

📌 Notice that Scala automatically infers the type `Int` and labels results like `res0` for unnamed expressions.

# Structure of a Scala Program

```
object ProgramName {  
  def main(args:Array[String]): Unit = {  
    // Statements  
  }  
}
```

| Component | Purpose                        |
|-----------|--------------------------------|
| object    | Defines a singleton object     |
| main()    | Starting point of the program  |
| args      | Command-line arguments         |
| Unit      | Indicates no value is returned |
| println() | Displays output to the screen  |

## Explaining HelloWorld

```
object HelloWorld {  
  def main(args:Array[String]):  
Unit = {  
    println("Hello Scala")  
  }  
}
```

**Output:** Hello Scala

- 1** `object HelloWorld`
  - Creates a **Singleton Object**.
  - Unlike Java, no need to instantiate an object explicitly.
- 2** `def main(args:Array[String]): Unit`
  - The entry point of execution.
  - `Unit` means no value is returned (equivalent to `void` in Java).
- 3** `println("Hello Scala")`
  - Prints text to the screen.
  - The simplest form of output in Scala.

# Expanded Function Format

Functions are reusable blocks of code. The general syntax in Scala is:

```
def functionName(parameter:type): returnType = {  
    statements  
}
```

**def**

Keyword to define a function

**Name & Parameters**

Function name followed by typed parameters

**Return Type**

Declared after the colon following parameters

**Body {}**

Contains the statements to execute

# Function Example 1: greet()

## Function Definition

```
def greet(): Unit = {  
    println("Welcome to  
Scala")  
}
```

## Function Call

```
greet()
```

## Output

```
Welcome to Scala
```

## Key Points

- Return type is `Unit` — no value returned
- Empty parentheses `()` indicate no parameters
- Called simply by writing the function name followed by `()`
- Useful for performing actions like printing output

# Function Example 2: add()

## Function Definition

```
def add(a:Int, b:Int): Int =  
{  
    return a + b  
}
```

## Function Call & Output

```
println(add(10, 20))  
// Output: 30
```

## Breaking It Down

- `a:Int, b:Int` — two integer parameters
- `Int` after `)` — return type is integer
- `return a+b` — explicitly returns the sum
- `{ }` — encloses the function body

Scala prefers an expression-oriented style — the last expression in a function body is automatically returned, so the `return` keyword is optional.

## With `return`

```
def add(a:Int, b:Int): Int =  
{  
  return a + b  
}  
println(add(10, 20))
```

## Without `return` (Preferred)

```
def add(a:Int, b:Int): Int =  
{  
  a + b  
}  
println(add(10, 20))
```

☞ Both produce the same result: `add(10, 20)` returns 30. The expression-oriented style is idiomatic Scala.

1

## Activity 1 – REPL Variable

Open REPL and  
execute: `val x = 100`

2

## Activity 2 – Square Function

```
def
square(n:Int):
Int = {
    n*n
}
square(5) //
Output: 25
```

3

## Activity 3 – Cube Function

```
def
cube(n:Int):
Int = {
    n*n*n
}
cube(3) //
Output: 27
```

By the end of this period, students should be able to:

- Explain the purpose and features of Scala
- Launch and use the Scala REPL interactively
- Write and execute a basic Scala program
- Understand the structure of a Scala program
- Define and call functions using the expanded function format



# Assignment

01

---

## Install & Verify

Install Scala and verify the version using `scala -version`.

02

---

## REPL Arithmetic

Execute three arithmetic expressions using the Scala REPL and record the outputs.

03

---

## Display Program

Write a Scala program to display:  
*Welcome to Scala Programming*

04

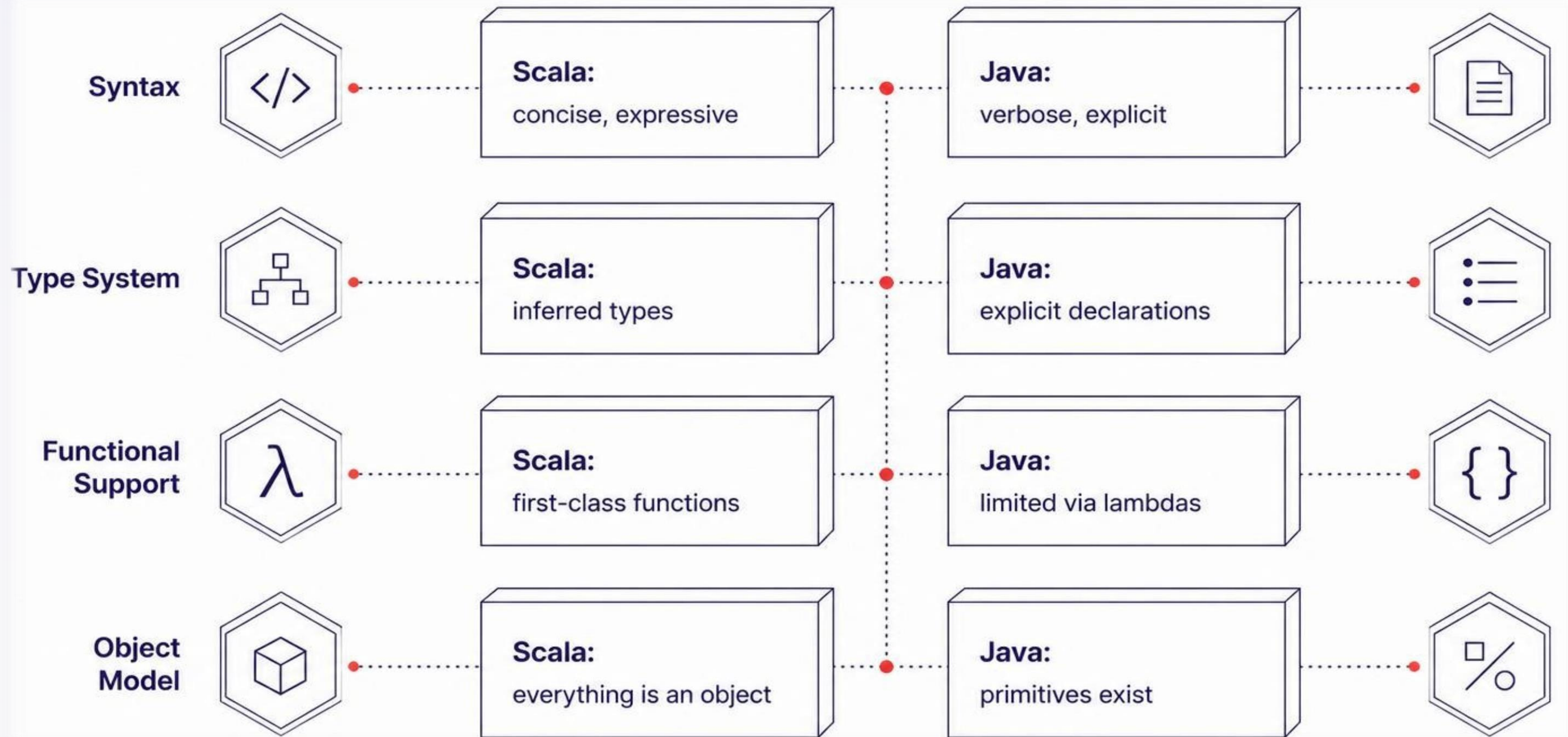
---

## Create Functions

Create functions for Addition, Subtraction, and Multiplication. Execute all in REPL and record outputs.

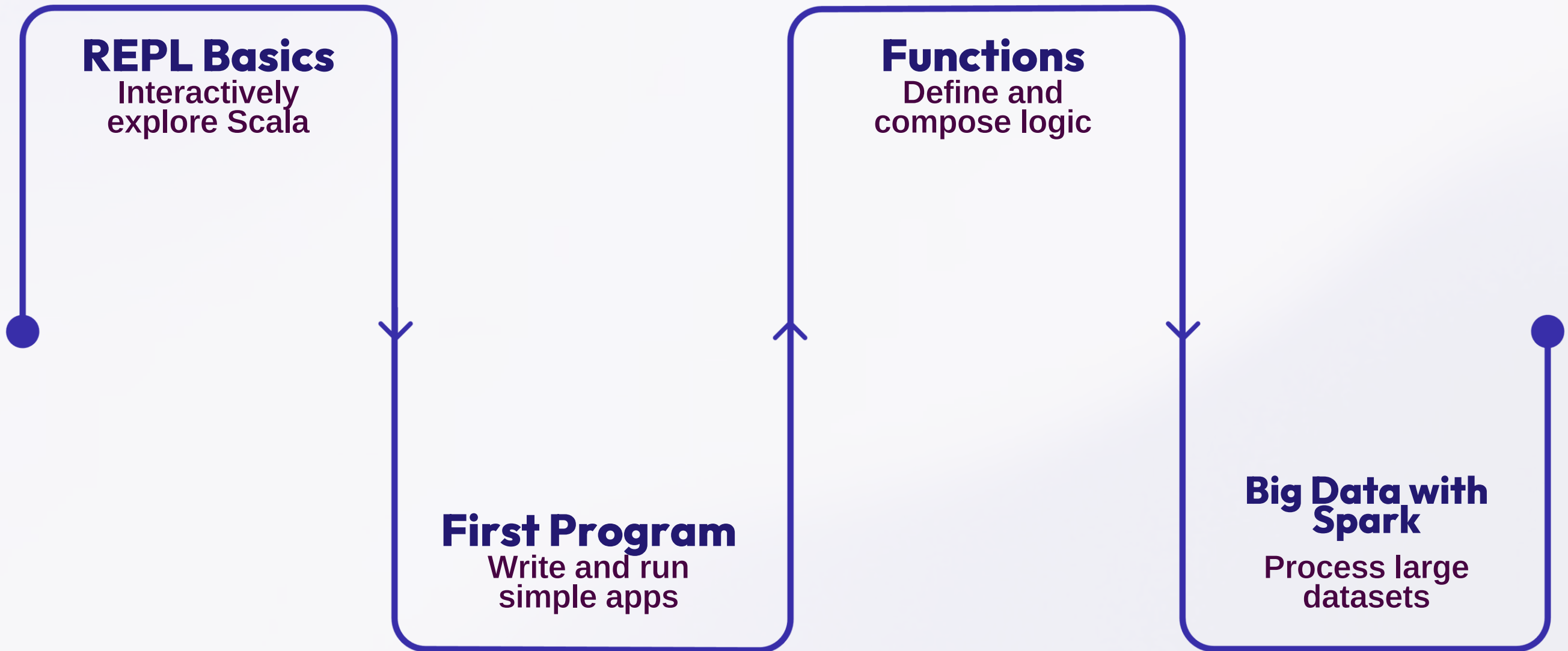


# Scala vs. Java: Quick Comparison



Scala's design choices make it more concise and expressive than Java while remaining fully interoperable on the JVM.

# Scala Learning Roadmap



This period covers the first two steps. Mastering REPL and basic programs builds the foundation for defining functions and eventually working with large-scale data frameworks like Apache Spark.